

Lista de Exercícios #6 – Métodos de Busca

Aluno: Thiago Henrique Gomes Feliciano - 1543790

Questão 01. Sobre os principais conceitos de métodos de busca, responda:

1. Um agente de resolução de problemas precisa ser capaz de formalizar qualquer situação antes de buscar uma solução. Quais são os cinco componentes exigidos para definir um problema de busca em IA? Defina e ilustre cada um com o exemplo do 8-puzzle.

R:

1. Estado inicial: É a configuração inicial do problema, ou seja, o ponto de partida da busca. No 8-puzzle, o estado inicial corresponde à disposição inicial das peças numeradas e do espaço vazio no tabuleiro 3x3.

2. Ações: São as operações que podem ser executadas em cada estado. No 8-puzzle, as ações consistem em mover o espaço vazio: para cima, para baixo, para esquerda e para direita. As ações possíveis dependem da posição atual do espaço vazio.

3. Modelo de transição: Define como um estado muda após a execução de uma ação. Em outras palavras, descreve o resultado de aplicar uma ação em determinado estado. No 8-puzzle, ao mover o espaço vazio para uma direção, ocorre a troca de posição entre o vazio e a peça adjacente.

4. Teste de objetivo: É a condição usada para verificar se a solução foi alcançada. No 8-puzzle, o objetivo normalmente é organizar as peças em ordem crescente.

5. Custo da Solução: Representa o custo total das ações executadas desde o estado inicial até o estado atual. No 8-puzzle, geralmente cada movimento possui custo igual a 1. Assim, o custo do caminho corresponde à quantidade total de movimentos realizados para chegar à solução.

Se uma solução usa 15 movimentos, então: $g(n)=15$

O objetivo dos algoritmos de busca é encontrar uma sequência de ações que leve ao estado objetivo com o menor custo possível.

2. Compare os algoritmos de Busca em Largura (BFS) e Busca em Profundidade (DFS) nos critérios de completude e otimalidade. Descreva um cenário ou problema em que a Busca em Profundidade poderia falhar (não terminar).

R: A completude é a capacidade do algoritmo de encontrar uma solução, se ela existir. A busca em profundidade(DFS) não é completa pois pode se perder em um ramo infinito ou muito profundo, mesmo que exista solução em outro ramo pode nunca chegar nela, a busca em largura(BFS) sempre encontra uma solução se ela existir, desde que o fator de ramificação seja finito.

A otimalidade é a capacidade de encontrar a melhor solução (menor custo ou menor profundidade). A BFS é ótima, pois encontra a solução com menor número de passos (nível mais raso). A DFS não é ótima, pode encontrar uma solução mais profunda mesmo existindo uma mais curta.

A DFS pode falhar em terminar situações como em uma árvore com profundidade infinita ou em loops.

3. A Busca de Custo Uniforme (equivalente ao algoritmo de Dijkstra) é uma generalização da BFS para grafos com custos variáveis nas arestas. Explique por que a BFS simples não é adequada quando as ações têm custos diferentes. Descreva como a Busca de Custo Uniforme garante encontrar a solução de menor custo. Qual estrutura de dados é fundamental para sua eficiência?

R: A Busca em Largura (BFS) funciona corretamente para encontrar o menor caminho apenas quando todas as ações possuem o mesmo custo, é para achar o caminho de menor custo não é útil, pois se tiver um caminho de 2 passos com custo 50 e um com 4 passos, mas custo 20, ele iria preferir o caminho de 2 passos mesmo sendo tendo maior peso. Para resolver esse problema utiliza-se a Busca de Custo Uniforme, que em vez de expandir os nós pela profundidade, ela sempre expande primeiro o nó com menor custo acumulado desde o estado inicial.

4. O que é uma função heurística $h(n)$ e como ela diferencia a Busca com Informação da Busca sem Informação? Cite pelo menos um exemplo de heurística abordada no material (por exemplo, no mapa da Romênia ou no 8-puzzle).

R: Uma função heurística $h(n)$ é uma estimativa do custo ou da distância restante entre um nó atual e o objetivo em um problema de busca. Ela é utilizada em algoritmos de busca com informação para orientar a exploração dos caminhos mais promissores, tornando a busca mais eficiente. Um exemplo é o mapa da Romênia, que tem como objetivo encontrar o caminho até Bucareste. A heurística utilizada é a distância em linha reta entre cada cidade e Bucareste. Assim, as cidades aparentemente mais próximas do destino recebem prioridade na exploração.

5. Tanto a Busca Gulosa quanto o A* utilizam funções heurísticas, mas com estratégias diferentes. Compare os dois algoritmos quanto às suas funções de avaliação $f(n)$, e demonstre com um exemplo (como o problema da viagem na Romênia) por que a Busca Gulosa pode encontrar soluções subótimas enquanto o A*, com heurística admissível, sempre encontra a solução ótima.

R: A Busca Gulosa considera apenas a estimativa do custo restante até o objetivo, $f(n)=h(n)$, ela escolhe sempre o nó que parece mais próximo do objetivo e ignora completamente o custo já percorrido. O algoritmo A* combina a estimativa de custo restante e o custo real do nó inicial até o nó final, $f(n)=g(n)+h(n)$.

No problema da Romênia, usa-se normalmente a distância em linha reta até Bucareste, que nunca é maior que a distância real pelas estradas. No caminho de Arad até Bucareste, na busca gulosa ele seguiria esse caminho seguindo o $h(n)$ que resultaria no caminho Arad→ Sibiu→ Făgăraș→ Bucareste, que teria um custo de $140+99+211=450$, mas o caminho

ótimo seria Arad→ Sibiu→ Rimnicu Vilcea→ Pitesti→ Bucareste, que teria um custo de $140+80+97+101=418$. O algoritmo A* nunca ignora um caminho potencialmente melhor, pois combina custo real e estimativa heurística, evitando decisões precipitadas.

6. Você está desenvolvendo um agente inteligente para três cenários distintos: (a) encontrar o caminho mais curto em um mapa de cidades onde as distâncias em linha reta são conhecidas; (b) jogar xadrez contra um oponente humano com limite de tempo por jogada; (c) resolver um labirinto desconhecido sem nenhuma informação prévia sobre o ambiente. Para cada cenário, indique qual estratégia de busca seria mais adequada (dentre BFS, DFS, Busca com Aprofundamento Iterativo, A*, MINIMAX com poda alfa-beta) e justifique sua escolha com base nas propriedades de cada algoritmo estudado.

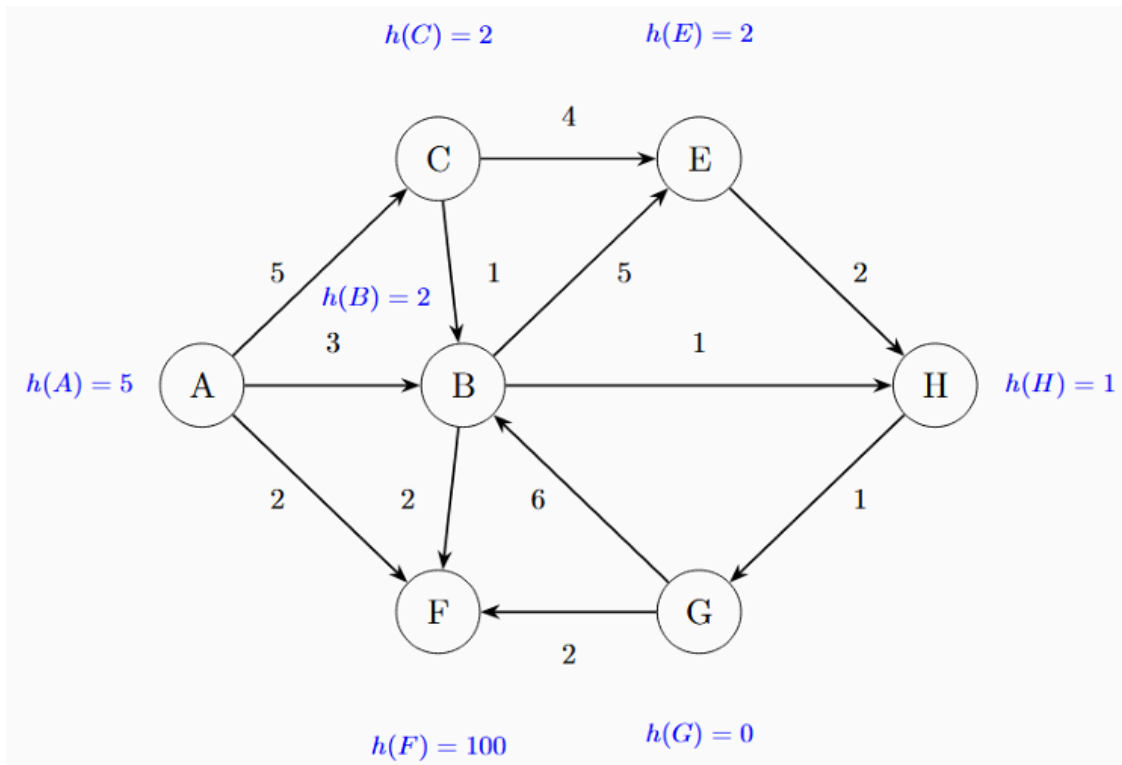
R:

a) Algoritmo A*, pois o algoritmo utiliza tanto o custo já percorrido quanto uma heurística que estima a distância restante até o objetivo, e como as distâncias em linha reta entre as cidades já são conhecidas, elas podem ser usadas como heurística, tornando a busca mais eficiente.

b) Algoritmo MINIMAX, pois o xadrez é um jogo competitivo de soma zero no qual um jogador tenta maximizar suas chances de vitória enquanto o outro tenta minimizá-las, e o MINIMAX analisa as possíveis jogadas futuras considerando que o adversário também fará as melhores escolhas possíveis e a poda alfa-beta melhora o desempenho do algoritmo ao eliminar ramos da árvore de busca que não afetarão a decisão final, permitindo analisar mais jogadas em menos tempo.

c) Busca com aprofundamento iterativo, pois esse método combina características da BFS e da DFS. Ele executa várias buscas em profundidade com limites crescentes até encontrar a solução.

Questão 02. Considere o problema de busca definido pelo seguinte grafo, onde A é o estado inicial e G é o estado objetivo, e o custo de cada ação é indicado na aresta correspondente.



Resolva o problema utilizando as seguintes estratégias de busca:

- Busca em largura (BFS)
- Busca em profundidade (DFS)
- Busca de custo uniforme
- Busca gulosa (o valor da heurística para cada nó n é dado por $h(n)$)
- A* (o valor da heurística para cada nó n é dado por $h(n)$)

Para cada estratégia, mostre a árvore de busca final, indicando o caminho encontrado de A a G e o custo da solução total. Considere a ordem de visitação em ordem alfabética.

R:

- Busca em largura (BFS):

Expansão(em níveis):

A → B, C, F

B → E, H

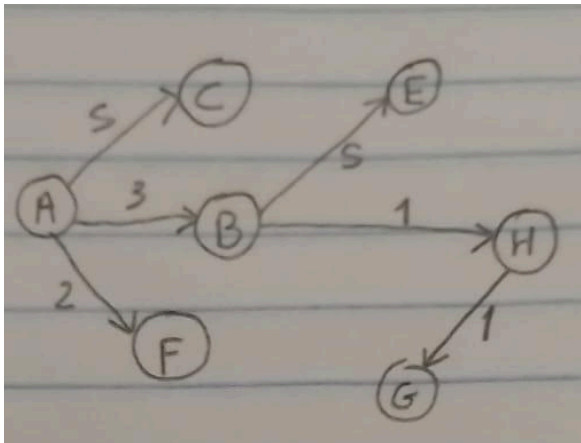
H → G

Caminho Encontrado:

A → B → H → G

Custo: $3+1+1 = 5$

Árvore de busca final:



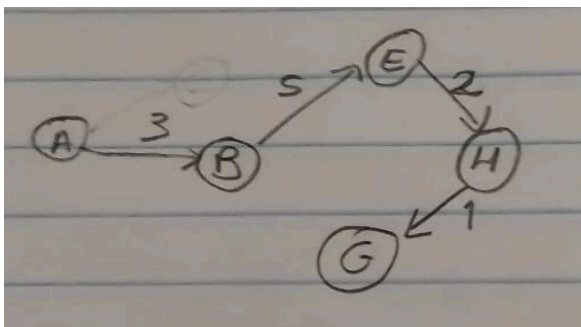
- Busca em profundidade (DFS):

Caminho Encontrado:

$A \rightarrow B \rightarrow E \rightarrow H \rightarrow G$

Custo: $3+5+2+1 = 11$

Árvore de busca final:



- Busca de custo uniforme:

Expansão(em níveis):

$A \rightarrow F, B, C$

$B \rightarrow H, E$

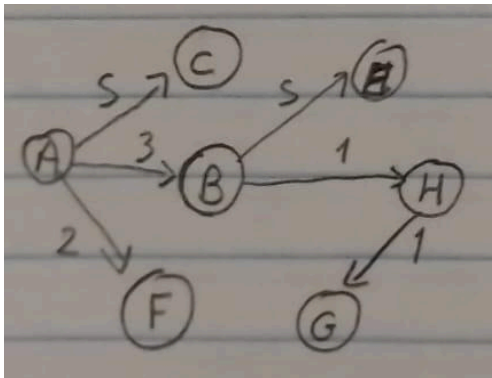
$H \rightarrow G$

Caminho Encontrado:

$A \rightarrow B \rightarrow H \rightarrow G$

Custo: $3+1+1 = 5$

Árvore de busca final:



- Busca gulosa (o valor da heurística para cada nó n é dado por $h(n)$):

Expansão(em níveis):

$A \rightarrow B$ (empate com C, escolhe B), C, F

$B \rightarrow H$ (escolhe H por ser menor), E

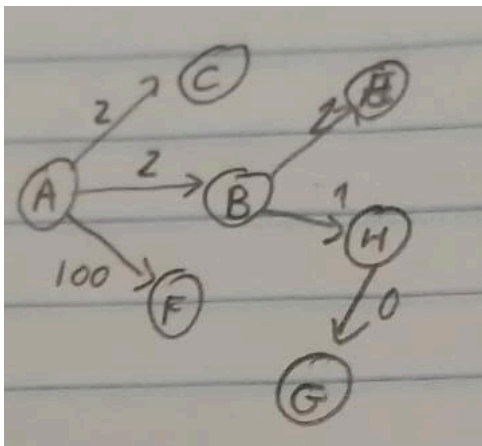
$H \rightarrow G$

Caminho Encontrado:

$A \rightarrow B \rightarrow H \rightarrow G$

Custo: $3+1+1=5$

Árvore de busca final:



- A^* (o valor da heurística para cada nó n é dado por $h(n)$):

Expansão(em níveis):

$A \rightarrow B(5), C(7), F(102)$

$B \rightarrow H(2), E(7)$

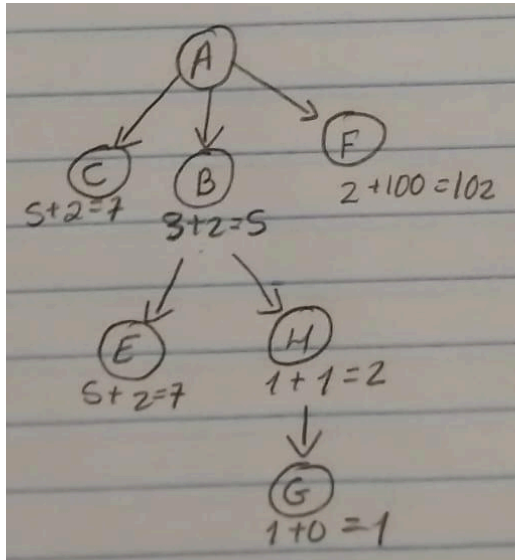
$H \rightarrow G(1)$

Caminho Encontrado:

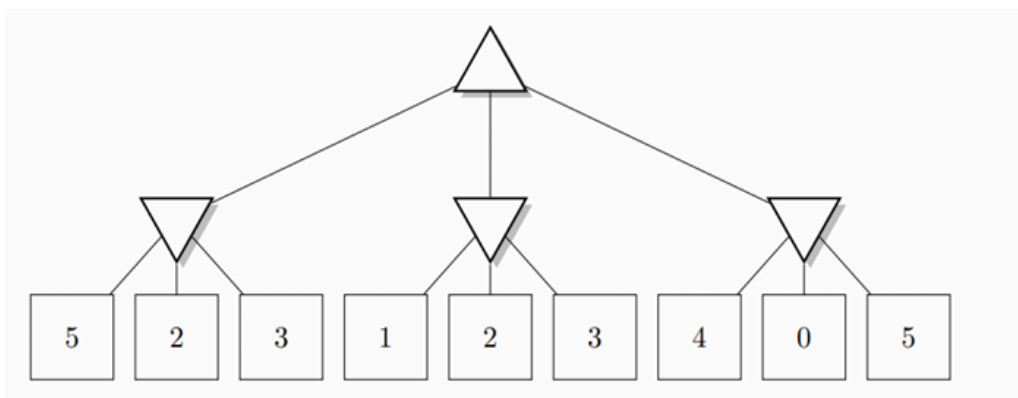
A → B → H → G

Custo: $3+1+1=5$

Árvore de busca final:



Questão 03. Considere a seguinte árvore representando um jogo de soma zero, onde os triângulos apontando para cima são nós MAX, os triângulos apontando para baixo são nós MIN e os quadrados são nós objetivos (terminais) com o valor correspondente da função de utilidade para o jogador MAX.



Aplique o algoritmo MINIMAX para encontrar a melhor ação para o jogador MAX na raiz. Mostre a árvore indicando o valor minimax para cada nó. 7

R:

